

SVS

Security Vulnerability Scanner System Documentation

System	SVS — Security Vulnerability Scanner
Version documented	Frontend badge v2.4 · build e474377
Date	22 September 2026
Source of truth	Code in backend/ and frontend/
Deployment model	Single-user, local, Docker Compose on a laptop
Companion files	README.md (quick start), /docs on port 8000 (live OpenAPI)

Contents

I. System overview

- What SVS is
- Design posture: no AI, no auth
- The thirty-second version
- Repository layout

II. Architecture

- Component topology
- Compose services
- Two contracts that make scanning possible
- Technology stack

III. Request lifecycle: a scan end to end

- Dispatch
- Ingestion
- Queue
- Worker execution
- Termination and cleanup
- Client polling

IV. Scan engines

- Shared container contract: `_run_container`
- Semgrep (SAST)
- Gitleaks (secrets)
- Dependency audit (SCA)
- Failure semantics

V. Findings normalization

VI. Scoring

- Formula and weights
- Worked examples
- Why hyperbolic, not flat

VII. API reference

- Conventions
- Scans
- Projects
- Root and health
- Derived response fields

VIII. Data model

- Tables
- Relationships
- Migration and startup
- The users table without authentication

IX. Frontend

- Application shell and state
- Views
- Service layer

- Adapter layer
- What the UI simulates
- Build and serving

X. Configuration

- Backend settings
- Environment file
- Frontend build configuration
- Rate limiting
- CORS

XI. Security posture

- Deliberate boundaries
- Honest exposures
- SSRF controls, client and server

XII. Testing

- Scoring unit tests
- Pipeline integration test
- Self-checks
- The vulnerable fixture

XIII. Operations

- `run-svs.bat` step by step
- The `localhost` shadow on port 3000
- Editing code: rebuild is mandatory
- Logs, stops, resets
- Database backup and restore

XIV. CI/CD pipeline

XV. Known limitations and documentation drift

XVI. File inventory

I. System overview

1. What SVS is

SVS is a security scanning service that ingests a Git repository or a `.zip` archive, runs three deterministic engines against it, and reports a security posture score with per-finding remediation guidance.

An SVS install is five Docker containers on one machine: a PostgreSQL database, a Redis broker, a FastAPI API, a Celery worker, and an nginx-served React frontend. The API accepts a target, the worker clones or unpacks it into a per-scan directory, and that directory is mounted read-only (or read-write, where a tool must write a report) into throwaway sibling scanner containers. Each scanner's findings are normalized into one common shape, written to the database, and scored. The frontend polls until the scan reaches a terminal state, then renders the findings with triage controls and a link to a server-generated PDF.

Aspect	Reality
Intended deployment	One operator's own laptop. Not a multi-tenant SaaS, despite some UI text inherited from the original spec.
Analysis type	Static: SAST patterns, secret detection, dependency CVEs. Nothing is executed, fuzzed, or dynamically probed.
AI involvement	None. Every finding comes from a deterministic tool; every score is arithmetic.
Authentication	Removed by design. See section I.2.
Where scanned code lives	On the host only, under <code>/tmp/svs_uploads/scan_<id></code> , deleted when the scan finishes.
Network egress	Only the Semgrep ruleset and the CVE advisory database are fetched. Scanned code never leaves the host.

2. Design posture: no AI, no auth

Two decisions are load-bearing and easy to misread:

- **No authentication.** Every request resolves to a single `local` user row (`backend/app/api/deps.py:11`). The users table survives only because `projects.owner_id` has a non-nullable foreign key to it, and because the historical projects and scans created under the old accounts are worth more than a clean schema. The row is created on first use with an unusable password (`!`) that no endpoint ever checks.
- **No inference.** The scoring service is a closed-form arithmetic function of finding counts. Two scans of the same code produce the same score, which is the point: the number can be reproduced from the report instead of trusted.

Concretely, no endpoint checks a token, no ownership filter is applied anywhere in the API, and the frontend renders a small green `local` pill in the navbar to say so. Every route is annotated `Auth: None (single-user local instance)` in the in-app API reference, and that is accurate.

3. The thirty-second version

1. The operator presses *Dispatch Security Pipeline* with either a Git URL, a zip archive, or the demo target.
2. The API validates the URL, clones or unpacks the code into `/tmp/svs_uploads/scan_<id>`, writes a `scans` row at `PENDING`, enqueues a Celery task, and returns immediately.
3. The worker marks the scan `RUNNING` and runs Semgrep, then Gitleaks, then a dependency audit, each in its own container with a memory cap and a 15-minute timeout.
4. Each tool's output is normalized into the same finding shape and inserted.
5. Scoring sums weighted risk points into `score = 100 / (1 + risk / 25)`.

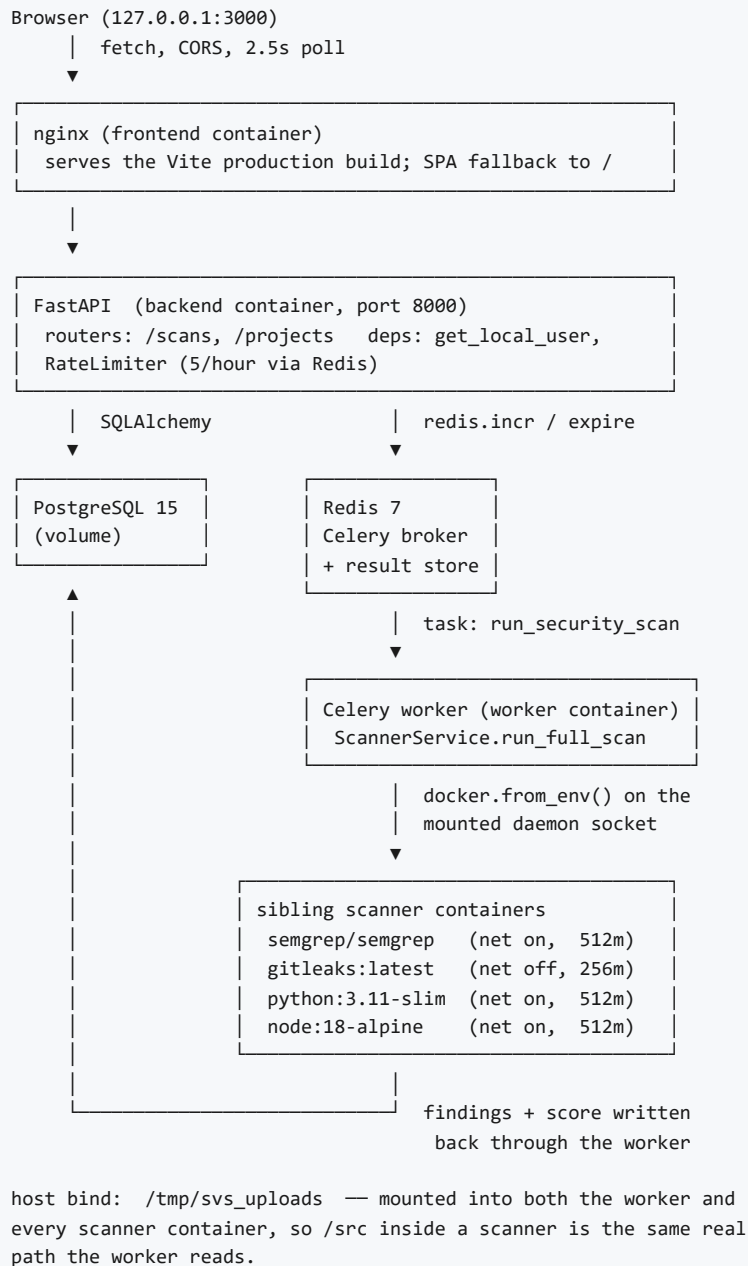
6. The worker deletes the scan directory, closes the session, and the scan is **COMPLETED** (or **FAILED** with the name of the broken scanner).
7. The dashboard's 2.5-second polling notices, and the report view renders findings, triage buttons, and a link to the server-side PDF.

4. Repository layout

```
SVS/
├── backend/                # FastAPI API + Celery worker (one image, two services)
│   ├── app/
│   │   ├── api/           # routers: scans, projects, deps
│   │   ├── core/         # config, celery app, rate limiting
│   │   ├── engine/       # ingestion, scanners, normalizer, scoring, reporting, tasks
│   │   ├── models/       # SQLAlchemy: user, project, scan, finding
│   │   ├── schemas/      # pydantic request/response models
│   │   ├── database.py    # engine + SessionLocal + get_db
│   │   └── main.py        # FastAPI app, CORS, startup migration
│   ├── tests/            # scoring unit tests + pipeline integration test
│   │   └── fixtures/vulnerable_project/
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/              # Vite + React 19 + TypeScript + Tailwind v4
│   ├── src/
│   │   ├── components/   # Navbar + 5 views
│   │   ├── data/         # static knowledge base + architecture docs
│   │   ├── services/     # api.ts (fetch), adapter.ts (mapping)
│   │   ├── types.ts
│   │   ├── App.tsx       # shell, state, polling
│   │   └── main.tsx
│   ├── nginx.conf
│   └── Dockerfile        # node build -> nginx
├── frontend-cra-legacy/   # superseded Create React App build, kept for reference
├── docs/                  # this document
├── docker-compose.yml
├── run-svs.bat            # the one-command launcher
├── start.bat              # bare up --build + open browser
├── .github/workflows/ci-cd.yml
├── .env                   # deployment configuration (not for reuse)
└── README.md
```

II. Architecture

1. Component topology



2. Compose services

From `docker-compose.yml`. All five use `restart: always`.

Service	Image / build	Ports	Volumes	Notes
db	postgres:15-alpine	5432:5432	postgres_data (named)	Healthcheck pg_isready, 5s interval, 5 retries. Backend and worker wait for it to be healthy.
redis	redis:7-alpine	6379:6379	-	No healthcheck; service_started only. Broker and result store for Celery, plus the rate-limit counter.
backend	build: ./backend	8000:8000	docker socket, /tmp/svs_uploads	Runs uvicorn on 0.0.0.0:8000. Hosts the API, the Swagger UI, and the PDF generator.

worker	build: ./backend	-	docker socket, /tmp/svs_uploads	Command: celery -A app.core.celery_app worker --loglevel=info. Same image as the backend; it needs the socket and the upload path, not a port.
frontend	build: ./frontend	3000:80	-	Build arg VITE_API_URL from FRONTEND_API_URL, default http://localhost:8000. nginx serves the static bundle.

The `frontend-cra-legacy/` directory contains the superseded Create React App build with its own Dockerfile and nginx config. It is not referenced by `docker-compose.yml` and does not run.

3. Two contracts that make scanning possible

Two non-obvious arrangements are what let the worker spawn scanner containers at all. Both are commented in the compose file.

3.1 The Docker socket mount

Both the backend and the worker mount `/var/run/docker.sock`. The scanners are not libraries or subprocesses; they are sibling containers started through the `docker` Python client (`docker.from_env()` in `engine/scanner.py:19`). The worker talks to the host daemon, so every scanner container is a peer of the worker, not a child of it. This is the single largest attack surface in the system and is documented honestly in section XI.2.

3.2 The host bind at `/tmp/svs_uploads`

This is a host bind, not a named volume, and the reason is mechanical. A scanner container receives `volumes=` `["/tmp/svs_uploads/scan_42:/src"]`. If `/tmp/svs_uploads` were a named volume, the path the worker passes would not resolve to the same real path on the daemon's filesystem, and the mount would be empty. The bind guarantees the worker and every scanner agree on one absolute path. It is also why uploads live under `/tmp` on the host rather than inside a container filesystem that would be discarded.

4. Technology stack

Layer	Choice	Pinned version
API framework	FastAPI on uvicorn	0.115.0 / 0.30.6
ORM	SQLAlchemy (declarative DeclarativeBase)	2.0.32
Database	PostgreSQL, driver psycopg2-binary	15-alpine / 2.9.9
Job queue	Celery, Redis broker and result store	5.4.0 / 5.0.3
Container control	docker Python SDK	7.1.0
Git cloning	GitPython (<code>Repo.clone_from</code> , <code>depth=1</code>)	3.1.43
PDF generation	xhtml2pdf (pisa) from an inline HTML template	0.2.16
Settings	pydantic + pydantic-settings	2.7.4 / 2.3.4
Multipart uploads	python-multipart	0.0.9
Frontend	React 19, Vite 8, TypeScript 7, Tailwind CSS v4 (via <code>@tailwindcss/vite</code>), lucide-react icons, motion	see <code>frontend/package.json</code>
Frontend serving	nginx stable-alpine with an SPA fallback	-

III. Request lifecycle: a scan end to end

This is the trace to read when something goes wrong. Every state transition below is a real line of code.

1. Dispatch

The frontend's `IngestScanView` collects a project name and either a Git URL or a file. Before calling the API it deduplicates projects: it calls `GET /projects/` and reuses an existing project with the same name, otherwise `POST /projects/` to register one. Every scan is keyed off a project row, and this avoids forking a duplicate project per scan. The demo path is different: `POST /scans/demo` clones a fixed target and files the result under its own `Demo Project` row, so no project is created or named by the client.

Three entry routes exist, all rate-limited at 5 scans per hour per user:

- `POST /scans/git` — multipart `project_id` + `repo_url`
- `POST /scans/upload` — multipart `project_id` + `file`, must end in `.zip`
- `POST /scans/demo` — no body; clones `https://github.com/juice-shop/juice-shop`

2. Ingestion

`IngestionService` (`backend/app/engine/ingestion.py`) writes the `scans` row first, at `PENDING`, so a scan ID exists before any work. Then:

- **Git** — `validate_repo_url` checks the scheme is `http/https/ssh/git`, requires a netloc, resolves the hostname through `getaddrinfo`, and rejects any resolved address that is private, loopback, link-local, multicast, or reserved. Then `Repo.clone_from(url, path, depth=1)`. A clone failure deletes the scan row and returns 400.
- **Zip** — reads the whole upload into memory and rejects anything over `MAX_FILE_SIZE` (50 MB), extracts into the scan directory, and on `BadZipFile` deletes the scan row and returns 400.

The scan directory is `<UPLOAD_DIR>/scan_<scan_id>`, i.e. `/tmp/svs_uploads/scan_42`. The scan ID determines the path, which is why the row must exist before extraction.

3. Queue

The router calls `run_security_scan.delay(scan.id, project.id)` and returns immediately with the scan ID and its `PENDING` status. The HTTP request is therefore bounded by ingestion time, not scan time. The task is registered as `tasks.run_security_scan`, `bind=True`, `max_retries=3`.

4. Worker execution

`run_security_scan` (`engine/tasks.py`) opens its own SQLAlchemy session (it does not share the API's), builds the project path, and calls `ScannerService.run_full_scan` (`engine/scanner.py:25`):

1. If the Docker client cannot be reached, the scan is marked `FAILED` and returns. This is the "Docker daemon is not available" path.
2. The scan is set to `RUNNING` and `start_time` is recorded.
3. Three scanners are constructed and run **in sequence**, not in parallel: Semgrep, then Gitleaks, then the dependency scanner. Sequence matters only for log readability; there is no data dependency between them.
4. Each scanner's exceptions are caught as `ScannerError` and collected into a `failures` list with the tool name and detail. A broken scanner does not abort the remaining ones.
5. `scoring_service.calculate_score(scan_id, db)` runs on whatever the surviving scanners found, so partial results are visible rather than discarded.
6. If any scanner failed, the scan is `FAILED` and `error_detail` holds the joined list, truncated to 500 characters. Reporting `COMPLETED` with a clean score would hide a broken pipeline as a clean repository.

7. Otherwise `COMPLETED`.

Any uncaught exception sets `FAILED` with a `Pipeline error:` prefix, and the task itself retries up to three times with a 60-second countdown.

5. Termination and cleanup

In the task's `finally`, the scan directory is removed with `shutil.rmtree` and the session is closed. This runs on success and on failure, which is why a finished scan leaves no source on disk and why the report view cannot show the offending line afterwards. The Gitleaks report file, which contains matched secrets in cleartext, is additionally deleted by the scanner itself before that.

6. Client polling

The dashboard polls `GET /scans/` every 2500 ms while any scan reports `status === 'scanning'` (the adapter collapses `PENDING` and `RUNNING` into `scanning`). The ingest view polls `GET /scans/<id>` every 2 s with a 20-minute deadline. Findings are only fetched on demand when a scan is selected, because loading every scan's findings up front would cost one call per scan on a dashboard that only needs counts.

IV. Scan engines

1. Shared container contract: `_run_container`

All three scanners subclass `BaseScanner` (`backend/app/engine/base_scanner.py`) and share one container runner. Its contract is the difference between a scanner that reports findings and one that hangs:

Property	Value
Execution	<code>detach=True</code> then <code>container.wait()</code> , never <code>detach=False</code> . Several tools exit non-zero on a <i>successful</i> scan; <code>detach=False</code> would raise before the report could be read.
Timeout	<code>DEFAULT_TIMEOUT = 900</code> (15 min) enforced by the caller's own deadline, not by the SDK. <code>container.wait()</code> raises <code>ReadTimeout</code> when its poll expires with the container still running; the loop re-arms and keeps waiting. On deadline the container is killed and a <code>ScannerError</code> is raised. This exists because a hung scanner used to block <code>wait()</code> forever, leaving the scan stuck on <code>RUNNING</code> and the Celery task wedged with no cleanup.
Memory	<code>mem_limit</code> per scanner (512 MB default, 256 MB for Gitleaks).
Network	<code>network_disabled</code> defaults to <code>True</code> ; only the scanners that need a ruleset or an advisory DB opt out.
Exit codes	<code>accept_exit_codes</code> extends the set that counts as success. Gitleaks and npm audit exit 1 to report <i>findings</i> , so their callers pass <code>(0, 1)</code> ; without that, the cleanest repo would scan as <code>FAILED</code> .
Output	Only stdout is returned. Most tools write the JSON report to stdout and progress chatter to stderr. Gitleaks is the exception and writes to a file.
Empty stdout	Returned as-is when the exit code is accepted: gitleaks and npm audit legitimately print nothing when the code is clean, and that is not a failure. Empty stdout with an unaccepted exit code raises <code>ScannerError</code> with the last 500 characters of stderr.
Cleanup	<code>container.remove()</code> in a finally, always.

Findings are persisted through `_save_finding`, which uppercases the severity and stores optional remediation text.

`ScannerError` carries a `tool` name so the pipeline can tell the operator *which* scanner broke.

2. Semgrep (SAST)

`backend/app/engine/semgrep_scanner.py`

Setting	Value
Image	<code>semgrep/semgrep</code> (unpinned latest tag)
Command	<code>semgrep scan --json --config=auto /src</code>
Mount	<code>/tmp/svs_uploads/scan_{id}:/src:ro</code> — read-only
Network	Enabled. <code>--config=auto</code> downloads the ruleset from <code>semgrep.dev</code> . Egress is for the ruleset only; the container never uploads scanned code.
Resources	<code>mem_limit 512m, cpu_period 100000, cpu_quota 50000</code> (50% of one CPU)
Exit codes	Default <code>(0,)</code>

Output is Semgrep's JSON. Each entry in `results` is normalized through `normalize_semgrep`; `check_id` becomes the vulnerability type, `extra.message` the description, `extra.remediation` the fix text.

3. Gitleaks (secrets)

`backend/app/engine/gitleaks_scanner.py`

Setting	Value
Image	zricethezav/gitleaks:latest (unpinned)
Command	detect --no-git --source /src --report-format json --report-path /src/.svs_gitleaks_report.json --no-banner
Mount	/tmp/svs_uploads/scan_<id>:/src — read-write, because the report is written inside the mount
Network	Disabled. Fully offline: entropy and pattern matching only.
Resources	mem_limit 256m
Exit codes	(0, 1) — exit 1 means leaks were found

Gitleaks streams nothing usable to stdout, so the report file is read back from the mount. If the file is absent or empty, the scan is clean and 0 is returned — not an error. The `finally` block deletes the report file, because it holds matched secrets in cleartext and should not outlive the scan directory. Every leak is normalized to vulnerability type `Secret-Leak`, severity `CRITICAL`, remediation "Revoke the secret immediately and rotate it."

4. Dependency audit (SCA)

`backend/app/engine/dependency_scanner.py` — language is detected from the project root: `requirements.txt` selects Python, `package.json` selects JavaScript, and neither means 0 findings.

4.1 Python path

Setting	Value
Image	python:3.11-slim
Command	sh -c "pip install --quiet pip-audit >/dev/null 2>&1 && pip-audit -r requirements.txt --format json"
Mount	:/src:ro
Network	Enabled, to fetch the CVE advisory database

The `-r requirements.txt` is load-bearing: with no arguments `pip-audit` audits the *container's* environment, not the project. Output is walked as `dependencies[].vulns[]`; each vulnerability becomes a `Dependency-Vulnerability` finding at `HIGH`, filed against `requirements.txt`.

4.2 JavaScript path

Setting	Value
Image	node:18-alpine
Command	sh -c "if [! -f package-lock.json]; then npm install --package-lock-only --ignore-scripts --no-audit --no-fund >/dev/null 2>&1; fi; npm audit --json"
Mount	:/src — read-write, so a missing lockfile can be generated
Network	Enabled, for the registry and the advisory feed

Three details in that command are deliberate and each has caused a failure before:

- `--ignore-scripts` means the scanner never executes the target repo's install or postinstall hooks — arbitrary code from untrusted input. It is also what makes repos with workspace-hopping postinstall scripts scannable at all.
- The separator is `;`, not `&&`, so a failed lockfile generation does not short-circuit the audit.

- `npm audit` exits 1 when it *finds* vulnerabilities, which is why the runner accepts exit code 1.

npm audit reports its own failures (missing lockfile, unreachable registry) as a JSON error object with exit 1 and empty vulnerability data. Without an explicit check that would read as "clean", so the scanner raises `ScannerError("SCA Audit", ...)` when `"error"` is present.

5. Failure semantics

Condition	Result
One scanner raises <code>ScannerError</code>	Others still run; findings from survivors are scored; scan is FAILED with <code>error_detail</code> naming the tool and reason
A scanner times out	Container killed, <code>ScannerError</code> , treated as above
A scanner exits non-zero with no stdout	<code>ScannerError</code> with stderr tail
Docker daemon unreachable	Scan set to FAILED immediately, no scanners attempted
Uncaught exception in the task	FAILED, task retried up to 3 times after 60 s
All scanners succeed, no findings	COMPLETED, score 100

V. Findings normalization

`FindingNormalizer` (`backend/app/engine/normalizer.py`) is the choke point where three unrelated tool vocabularies collapse into one. Scoring, the findings API, and the PDF all consume that one shape, so a mapping bug would silently poison all three rather than break anything visibly.

The severity maps

```
SEMGREP_SEVERITY = {"ERROR": "HIGH", "WARNING": "MEDIUM", "INFO": "INFO"}
NPM_SEVERITY     = {"critical": "CRITICAL", "high": "HIGH",
                    "moderate": "MEDIUM", "low": "LOW", "info": "INFO"}
```

- Semgrep reports `ERROR` / `WARNING` / `INFO` ; SVS scores `CRITICAL` / `HIGH` / `MEDIUM` / `LOW` / `INFO` . There is no Semgrep level that maps to `CRITICAL` .
- npm reports `moderate` , not `medium` . A naive `.upper()` produced a severity the scoring weights do not recognize, and every moderate advisory silently scored zero and counted toward nothing. The explicit map fixes that.
- Unknown values fall back to `MEDIUM` , never to zero, at every level: the normalizer, the scorer's `WEIGHTS.get(severity, 4)` , and the severity counter.

Output shape

Every normalizer returns the same dict, consumed as `**kwargs` by `_save_finding` :

Field	Source per tool
<code>v_type</code>	Semgrep: the rule's <code>check_id</code> . Gitleaks: literal <code>Secret-Leak</code> . Both SCA paths: literal <code>Dependency-Vulnerability</code> .
<code>severity</code>	Semgrep and npm: mapped as above. Gitleaks and pip-audit: constant <code>CRITICAL</code> and <code>HIGH</code> respectively.
<code>file_path</code>	Semgrep: the affected file. Gitleaks: the file containing the secret. SCA: <code>requirements.txt</code> or <code>package.json</code> .
<code>line</code>	Semgrep: <code>start.line</code> . Gitleaks: <code>StartLine</code> . SCA: <code>0</code> — a manifest has no single offending line.
<code>desc</code>	The rule message, the rule ID, or the vulnerable package and advisory id.
<code>remediation</code>	The rule's own remediation text, or a fixed string per tool (rotate the secret; update the package; run <code>npm audit fix</code>).

Tool attribution is derived, not stored. The findings API infers the source tool from `vulnerability_type` : `Secret-Leak` → Gitleaks, `Dependency-Vulnerability` → "SCA Audit", anything else → Semgrep (`backend/app/api/scans.py:147`). This is correct only while the three scanners keep those exact type values; a `scanner_tool` column would be needed if that changes.

The module ships with an executable self-check (`python -m app.engine.normalizer`) asserting the severity map, including that a finding with no severity field still lands in a scored bucket.

VI. Scoring

1. Formula and weights

backend/app/engine/scoring.py

```
WEIGHTS = {"CRITICAL": 10, "HIGH": 7, "MEDIUM": 4, "LOW": 1, "INFO": 0}
DEDUCTION_SCALE = 25.0

risk    = sum(WEIGHTS[f.severity] for f in findings)  # unknown severity -> 4
score   = round(100 / (1 + risk / 25))
```

Info findings count zero. The scan row stores the score plus the four severity counts, which is what the dashboard aggregates without a join.

2. Worked examples

Findings	Risk	Score
none	0	100
1 Low	1	96
1 Medium	4	86
1 High	7	78
1 Critical	10	71
1 Crit + 1 High + 1 Med + 1 Low	22	53
2 × Info	0	100
2 × unknown severity	8 (falls back to Medium)	76
10 × Critical	100	20
40 × Medium	160	14
80 × Medium	320	7

These match the assertions in `tests/test_scoring.py` exactly.

3. Why hyperbolic, not flat

A flat `100 - deduction` saturates at 0. A repo with 25 medium findings and one with 300 both score 0 and stop ranking anything. The hyperbolic discount keeps 100 for a clean repo, never reaches 0, and stays strictly discriminative at every scale: 40 mediums score 14, 80 mediums score 7, so a worse repo is always a lower number. The documented ceiling is that no score can ever hit 0, only approach it — which is the intended behaviour.

The frontend mirrors this: the dashboard and report view both print `score = 100 / (1 + risk/25)` and the weight table next to the number, and the report view grades A ≥ 90, B ≥ 80, C ≥ 70, D ≥ 60, F below.

The scoring module has a `__main__` self-check using fake query objects that asserts: clean is 100, 40 medium > 80 medium in score, critical < high < low, unknown severity equals medium, and mixed-case severities match uppercase.

VII. API reference

FastAPI, served on `http://127.0.0.1:8000`. Live Swagger at `/docs`, OpenAPI at `/openapi.json`. Every route requires no authentication and applies no ownership filter. Scan-triggering routes are rate-limited.

1. Conventions

- JSON for `/projects` bodies; `multipart/form-data` for the scan routes, because they carry files and form fields together.
- Errors are FastAPI's `{"detail": "..."};` the frontend's client prefers that field and falls back to the status text.
- `GET /scans/` and `GET /scans/{id}` return *different* shapes: the list uses `critical_count`, the detail uses flat `critical`. The frontend keeps two interfaces for this reason.

2. Scans

POST `/scans/git`

Purpose	Clone a remote repository and queue a scan. Returns immediately.
Request	multipart/form-data: project_id (int, required), repo_url (string, required)
Response 200	<code>{"scan_id": 42, "status": "PENDING", "message": "Repository cloned. Scan queued in background."}</code>
Errors	404 project not found · 400 unsupported protocol / invalid URL / unresolvable hostname / internal address blocked / clone failure · 429 rate limit

POST `/scans/upload`

Purpose	Accept a zip archive against an existing project and queue a scan.
Request	multipart/form-data: project_id (int, required), file (File, required)
Response 200	<code>{"scan_id": 43, "status": "PENDING", "message": "Code ingested. Scan queued in background."}</code>
Errors	404 project not found · 400 filename does not end in .zip / size over 50 MB / invalid zip archive · 429 rate limit

Frontend vs backend on archives. The upload UI accepts `.zip` and `.tar.gz` and validates both client-side; the API accepts only `.zip`. A `.tar.gz` passes the client and is rejected by the server with 400.

POST `/scans/demo`

Purpose	Queue a scan of the deliberately vulnerable Juice-Shop repository. Creates the Demo Project row on first call and reuses it afterwards.
Request	none
Response 200	<code>{"scan_id": 44, "status": "PENDING", "message": "Demo scan queued for Juice-Shop."}</code>
Errors	429 rate limit · 400 if the clone fails

GET `/scans/`

Purpose	List every scan newest-first, with score and severity counts. Drives the dashboard's aggregates in one call.
Response	<code>ScanOut[]: id, project_id, status, created_at, score, error_detail, critical_count, high_count, medium_count, low_count</code>

GET `/scans/{scan_id}`

Purpose	Poll one scan: status, score, flat severity counts, runtime, error detail.
Response	id, status, score, project_id, created_at (ISO), critical, high, medium, low, duration, error_detail
Errors	404 scan not found

GET `/scans/{scan_id}/findings`

Purpose	List the normalized findings for a scan.
Response	Array of id, severity (capitalized), title, description, remediation, file, line, tool, status
Notes	tool is derived from title; status is the persisted triage state, defaulting to open

PATCH `/scans/{scan_id}/findings/{finding_id}`

Purpose	Persist a triage decision so it survives a reload.
Request	application/json: {"triage": "resolved"}
Allowed values	open, resolved, false_positive, accepted
Response 200	{"id": 7, "scan_id": 42, "status": "resolved"}
Errors	400 value outside the allowed set · 404 finding not found under that scan

Why this endpoint exists. Triage used to be client-only: the buttons worked, then vanished on reload, which made them worse than absent. The `triage_status` column and this route were added together.

DELETE `/scans/{scan_id}`

Purpose	Delete a scan and every finding attached to it.
Response	204 no content
Errors	404 scan not found
Notes	Findings go first, then the scan: the FK has no ON DELETE CASCADE, so deleting the scan row alone used to raise a 500.

GET `/scans/{scan_id}/export`

Purpose	Generate and stream a PDF report for the scan.
Response	application/pdf, filename scan_report_{scan_id}.pdf
Content	Project name, overall score with the formula printed beneath it, severity distribution table, and one block per finding with file, line, triage state, and remediation
Errors	404 scan not found · 500 if PDF generation fails

The PDF is rendered from an inline HTML template by `xhtml2pdf` (`backend/app/engine/reporting.py`) into a named temp file that is streamed and left behind in `/tmp`. The frontend links to it with a plain anchor — no auth, no blob handling.

3. Projects

POST `/projects/`

Purpose	Register a project so scans can be attached to it.
Request	<code>application/json: {"name": "billing-service", "repo_url": "https://github.com/org/repo"}</code>
Response 201	<code>ProjectOut: id, name, repo_url, owner_id</code>

GET `/projects/`

Purpose	List all projects, newest id first. The New Scan form reads this to avoid creating duplicate projects.
Response	<code>ProjectOut[]</code>

GET `/projects/{project_id}`

Purpose	Fetch one project.
Response	<code>ProjectOut</code> · 404 if not found

GET `/scans/project/{project_id}`

Purpose	List the scans for one project, newest first.
Response	<code>ScanOut[]</code> · 404 if the project does not exist

DELETE `/projects/{project_id}`

Purpose	Delete a project together with all of its scans and findings.
Response	204 no content
Notes	Same CASCADE-absence fix as scan deletion: findings for the project's scans are deleted, then the scans, then the project.

4. Root and health

Route	Response
GET <code>/</code>	<code>{"message": "Welcome to the SVS API", "status": "online"}</code>
GET <code>/health</code>	<code>{"status": "healthy"}</code> — used by the launcher's readiness probe
GET <code>/docs</code>	Swagger UI, auto-generated from the routes above

5. Derived response fields

Two API responses contain values computed on the way out rather than stored:

- `tool` on findings — derived from `vulnerability_type`.
- `status` on findings — `triage_status` if set, else the string `"open"`; the column itself is `NOT NULL DEFAULT 'open'` so this is only observable on rows predating the migration.

VIII. Data model

Four tables, PostgreSQL 15, SQLAlchemy 2.0 declarative. Integer primary keys. No foreign key carries `ON DELETE CASCADE`, which is why the delete endpoints remove children explicitly.

1. Tables

users

Column	Type	Constraints
id	Integer	primary key, indexed
username	String	unique, indexed, not null
email	String	unique, indexed, not null
hashed_password	String	not null (unusable noise: ! for the local row)
created_at	DateTime	default utcnow

projects

Column	Type	Constraints
id	Integer	primary key, indexed
name	String	not null
repo_url	String	not null — for an upload, the archive filename
owner_id	Integer	FK users.id, not null
created_at	DateTime	default utcnow

scans

Column	Type	Constraints
id	Integer	primary key, indexed
project_id	Integer	FK projects.id, not null
status	String	not null; PENDING, RUNNING, COMPLETED, FAILED
created_at	DateTime	default utcnow
start_time	DateTime	default utcnow; set to the real start when the worker marks RUNNING
end_time	DateTime	nullable; set in the worker's finally
duration_seconds	Float	nullable; derived from the two timestamps
score	Integer	default 100
critical_count ... low_count	Integer	default 0; four columns
error_detail	String	nullable; joined scanner failures or pipeline error, truncated to 500 chars

findings

Column	Type	Constraints
--------	------	-------------

id	Integer	primary key, indexed
scan_id	Integer	FK scans.id, not null
vulnerability_type	String	not null; e.g. python.sql-injection, Secret-Leak
severity	String	not null; INFO, LOW, MEDIUM, HIGH, CRITICAL
file_path	String	not null
line_number	Integer	nullable; 0 for dependency findings
description	Text	not null
remediation	Text	nullable; the report view falls back to a fixed string
triage_status	String(16)	not null, default 'open'; added by startup migration
created_at	DateTime	default utcnow

2. Relationships

```
users 1—* projects      (owner / back_populates "projects")
projects 1—* scans      (project / back_populates "scans")
scans 1—* findings      (scan / back_populates "findings")
```

`models/__init__.py` imports every mapper so the string-based relationship targets (`"Project"`, `"Finding"`) resolve regardless of which module a process enters through — API, worker, or a standalone script.

3. Migration and startup

Schema creation is deliberately primitive and happens at import time in `app/main.py`:

1. `Base.metadata.create_all(bind=engine)` creates all four tables on a fresh database and is a no-op on an existing one.
2. Because `create_all` will not `ALTER` an existing table, the one schema change since the initial release is applied explicitly and idempotently: `ALTER TABLE findings ADD COLUMN IF NOT EXISTS triage_status VARCHAR(16) NOT NULL DEFAULT 'open'`.

There is no Alembic. Adding a column means adding a guarded `ALTER` here.

4. The users table without authentication

Auth was removed because SVS runs on the operator's own laptop and is not exposed publicly. The table survives for the FK and for the historical rows. The resolution function is deterministic: every request in the process's lifetime resolves to the same `local` user, so new projects get a stable owner and the rate limiter has one key. The `UserCreate` / `UserOut` schemas still exist in `schemas/user.py` but no route uses them.

Documentation drift. The frontend's *Architecture Docs* → *Database Schema (ERD)* panel describes a schema that does not exist: UUID primary keys, a `role` column, JSONB severity counts, a `scanner` column, and "Prisma ORM". The real schema is the one above. That panel is stale spec text, not a description of the shipped database, and should not be trusted. See section XV.

IX. Frontend

Vite + React 19 + TypeScript, Tailwind CSS v4, lucide-react icons. Five views, one stateful shell, no router library and no global store: navigation is a `activeTab` string in `App.tsx`.

1. Application shell and state

`frontend/src/App.tsx` holds:

- `scans` and `projects`, loaded together in one `Promise.all` and mapped through the adapter.
- `activeScanId`, with `activeScan` falling back to the first row so the report tab has something to show.
- A polling effect: while any scan is `scanning`, a 2500 ms interval re-runs the load. The interval is cleared when nothing is in flight.
- Handlers for select-scan (fetches findings on demand), scan-complete, delete-scan, and triage update with an optimistic apply and a rollback on error.

Findings are attached to a scan object only when that scan is opened. This keeps the dashboard to two list calls regardless of how many scans exist.

2. Views

View	What it shows	What it does
DashboardView	Mean posture score with a letter grade, total critical and high counts across completed scans, scan count and how many passed at 100; a filterable scan list with per-scan severity pills and score badge	Search by project or source name; filter by status (complete / scanning) and by severity (critical / clean); open a report; delete a scan; refresh
IngestScanView	Three ingestion modes: zip upload (drag-and-drop), Git URL, and the demo target; then a progress page with a five-stage indicator and a simulated terminal log	Validates extension and 50 MB size; runs a client-side SSRF guard list; deduplicates projects; dispatches the scan; polls <code>GET /scans/{id}</code> every 2 s up to 20 minutes; routes to the report on completion or surfaces <code>error_detail</code> on failure
ScanReportView	Score gauge with grade and label, severity metrics with per-severity risk weights, filterable findings list; each finding expands to description, location, remediation, and triage buttons	Filter by severity, source tool, and triage state; search by type, file, CWE, or CVE; mark resolved / false positive / accept risk; export JSON (client-built) or the server PDF (anchor link)
KnowledgeBaseView	Static dataset of 8 vulnerability classes with CWE ids, exploitation impact, vulnerable vs secure code, and a remediation checklist	Search and filter by SAST / Secrets / Dependencies; copy any code block
ArchitectureDocsView	System architecture overview with a Mermaid topology, the 15-step build prompts, an API reference, and an ERD panel	Four sub-tabs; copy prompt text; download a project zip

3. Service layer

`frontend/src/services/api.ts` is a thin fetch wrapper. It resolves `API_BASE_URL` from `import.meta.env.VITE_API_URL`, defaulting to `http://localhost:8000`. Its `request` helper unwraps FastAPI's `detail` field into a thrown `Error` carrying a `status`, treats 204 as `undefined`, and deliberately omits a Content-Type on multipart requests so the browser sets the boundary itself. `exportUrl` is a plain string, not a fetch, because the PDF endpoint needs no auth and returns a file.

4. Adapter layer

`frontend/src/services/adapter.ts` exists because the API and the UI do not share a vocabulary:

Mapping	Rule
Status	COMPLETED → complete, FAILED → failed, everything else (including PENDING) → scanning
Severity	lowercased and validated against the four known values, else low
Tool	Gitleaks → gitleaks, SCA Audit → dependency-audit, else semgrep
Source type	Derived from the project's <code>repo_url</code> via a regex: a real URL means cloned, a bare filename means uploaded. The scan row carries no source-type field.
Counts	The list and detail shapes (<code>critical_count</code> vs <code>critical</code>) are normalized into one counts object with a <code>total</code>

It also sets the fields the API does not have: `snippet` and `secureCodeExample` are always empty strings, because the scanned source tree is deleted when the worker finishes. The report view hides those panels when empty and explains why instead.

5. What the UI simulates

The stage indicators and terminal log lines are an approximation. The backend reports only `PENDING` / `RUNNING` / `COMPLETED` / `FAILED`; there is no per-stage status table. The ingest view advances its five stages on a timer while the worker is in flight and snaps to the real state when the status lands. The log lines are static strings describing what each stage does, not streamed tool output. The `Branch / Tag / Commit` field is advisory only: the backend has no branch column and always clones the default branch at depth 1. Per-stage status would need a stage table in the API.

6. Build and serving

The Dockerfile is a two-stage build: `node:20-alpine` runs `npm install` and `npm run build`, then the `dist` output is copied into `nginx:stable-alpine`. `VITE_API_URL` is a build argument, not a runtime variable, because Vite only exposes `VITE_`-prefixed vars to the bundle; the API origin is baked into the JavaScript. The compose file passes `FRONTEND_API_URL` in as that build arg.

`nginx.conf` has two behaviours worth knowing: static assets get `expires 1y` with an immutable cache header and a real 404 on a missing file (serving HTML for a missing `.js` would make the browser run HTML as JavaScript and the app would fail to boot with a syntax error); everything else falls back to `index.html` for the SPA. The dev server runs on port 3000 with `--host=0.0.0.0`, which conflicts with the compose frontend, so local development requires stopping the container first.

X. Configuration

1. Backend settings

`backend/app/core/config.py` — a `pydantic-settings` `BaseSettings` class reading `.env`:

Setting	Default	Notes
PROJECT_NAME	"Security Vulnerability Scanner"	Not rendered anywhere in the UI
DATABASE_URL	postgresql://svs_user:svs_password@db:5432/svs_database	Overridden by compose to use the <code>.env</code> credentials
UPLOAD_DIR	/tmp/svs_uploads	Must match the compose host bind; the scanners' mounts are built from it
CORS_ORIGINS	"http://localhost:3000"	Comma-separated; compose sets both spellings
REDIS_HOST	"redis"	Service name
REDIS_PORT	6379	-

2. Environment file

`svs/.env` supplies the compose-level variables:

Variable	Value	Used by
SECRET_KEY	64-char hex string	Left over from the removed JWT auth; nothing reads it
CORS_ORIGINS	http://localhost:3000	Overridden in compose by the two-spelling list
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	svs_user / svs_password / svs_database	The db service and both DATABASE_URL overrides
FRONTEND_API_URL	http://localhost:8000	Baked into the frontend bundle at build time

The file carries its own warning that it must be regenerated per deployment and never committed. It is listed in `.gitignore`.

3. Frontend build configuration

- `vite.config.ts` — React and Tailwind plugins, an `@` path alias to the project root, and an HMR/watch override driven by `DISABLE_HMR` for use in AI-assisted editors where file watching causes flicker.
- `package.json` — `lint` is `tsc --noEmit`. Dependencies include `motion` and `@google/genai`, which the current views do not import; they are carried over from the template the app was scaffolded from.

4. Rate limiting

`backend/app/core/rate_limit.py` and the `RateLimiter` dependency in `api/deps.py`:

- 5 requests per rolling 1-hour window per user, applied to `/scans/git`, `/scans/upload`, and `/scans/demo` only.
- Implemented with a Redis pipeline: `INCR` then `TTL`, and `EXPIRE` set only when the TTL reads `-1` (a key with no expiry), which is the first request of a window.

- Overrun raises 429 with a message naming the limit. The counter is keyed `rate_limit:user_<id>`, and because there is one user there is one key.

5. CORS

Origins come from `settings.CORS_ORIGINS` so a deployment does not need a code change to allow its own domain; an empty list still permits same-origin requests. Compose sets `http://localhost:3000,http://127.0.0.1:3000` because CORS compares origin strings and a browser reaching the app via either spelling must be allowed.

XI. Security posture

SVS runs untrusted code analysis on a laptop. Some boundaries are careful and deliberate; some exposures are accepted by the single-user deployment model. Both are listed, because a reader needs to know which is which.

1. Deliberate boundaries

- **Least-egress containers.** Every scanner runs in its own container with a memory cap (256–512 MB) and a 15-minute timeout. Gitleaks runs with no network at all; Semgrep and the CVE audit egress only to fetch the ruleset and the advisory database.
- **Read-only mounts where possible.** Semgrep and pip-audit mount `/src:ro`. Only Gitleaks and npm mount read-write, and only because they must write a report or generate a lockfile.
- **No execution of target code during install.** `--ignore-scripts` on every npm invocation means the scanner never runs the target repo's install or postinstall hooks — arbitrary code from untrusted input.
- **SSRF validation server-side.** Scheme allowlist, netloc required, hostname resolved, and every resolved address rejected if private, loopback, link-local, multicast, or reserved. Unresolvable hostnames and unparseable addresses are rejected, not passed through. Cloud metadata (`169.254.169.254`) is covered by the link-local rule.
- **SSRF guard client-side.** The Git URL field rejects localhost, RFC 1918 ranges, the metadata endpoint, and `file://` / `gopher://` before the request is made, with an inline warning.
- **Size ceiling.** 50 MB on uploads, enforced in the frontend, the ingestion service, and the multipart body.
- **Ephemeral storage.** The scan directory is `rmtree`'d in a `finally` on success and failure. The Gitleaks report, which contains matched secrets in cleartext, is deleted separately before that.
- **Scanned code never leaves the host.** Only the ruleset and the advisory database traverse the network.
- **Rate limiting.** 5 scans per hour per user.

2. Honest exposures

These are real. They are accepted by the deployment model, not mitigated.

- **The Docker socket is mounted into the backend and worker containers.** Anything that can reach the API can start arbitrary containers on the host with arbitrary mounts. This is how the scanners work; it is also full host compromise if the API is exposed.
- **No authentication.** There is no token, session, or ownership check anywhere. If port 8000 is reachable, it is fully usable.
- **PostgreSQL is published on 5432** with the credentials in `.env` (`svs_password`), which are the same ones used for local development and are not secret in any meaningful sense.
- **The `SECRET_KEY` in `.env` is a leftover from the removed JWT authentication** and is read by nothing.
- **Scanner images are unpinned** (`semgrep/semgrep`, `zricethezav/gitleaks:latest`, `node:18-alpine`). A hostile or compromised upstream tag would run inside the sandbox. Only `python:3.11-slim` and the Postgres/Redis images are version-anchored by the compose file.
- **Zip extraction is not path-traversal hardened.** `ZipFile.extractall` honours absolute and `..` paths in recent Python, but a hostile archive is still the threat model this scanner is built to analyze, so treat untrusted zips as untrusted input.

The conclusion to write down: SVS is safe to run on your own laptop against code you already have, and unsafe to expose on a network.

3. SSRF controls, client and server

The two checks are independent and both run. The client's regex list rejects obvious internal spellings and gives immediate feedback; the server's DNS-resolution check is the one that actually matters, because it catches a hostname that resolves to an internal address regardless of how it is spelled. A public hostname with an internal A record is caught server-side and silently passes the client.

XII. Testing

1. Scoring unit tests

`backend/tests/test_scoring.py` — six tests against an in-memory SQLite database, with `expected_score` recomputed from the same weights so the test cannot just restate the implementation:

Test	Assertion
<code>test_calculate_score_no_findings</code>	Clean repo scores exactly 100
<code>test_calculate_score_only_info</code>	INFO findings cannot move the score: still 100
<code>test_calculate_score_mixed_findings</code>	1 each of crit/high/med/low = 22 risk → 53, and all four count columns are set
<code>test_calculate_score_never_reaches_zero</code>	10 criticals (100 risk) → 20, not 0
<code>test_score_stays_discriminative_at_scale</code>	0 < 80 mediums < 40 mediums < 100
<code>test_calculate_score_unknown_severity_falls_back_to_medium</code>	UNKNOWN and NONE score as 2 mediums → 76
<code>test_calculate_score_is_monotonic</code>	Across seven severity profiles, more risk is always a strictly lower score

2. Pipeline integration test

`backend/tests/test_pipeline.py` runs the real `ScannerService.run_full_scan` against the fixture and asserts all three engines fired: Gitleaks found a secret at high/critical, pip-audit found a vulnerable dependency in `requirements.txt`, and Semgrep found a code-level vulnerability. It requires the scanner images to be present, so it runs against Docker, not SQLite — the database is SQLite, but the containers are real.

3. Self-checks

Two modules carry executable `__main__` blocks that need no test harness:

- `python -m app.engine.scoring` — clean is 100, the saturation property holds, severity ordering holds, unknown severity equals medium, case-insensitivity holds.
- `python -m app.engine.normalizer` — the severity map, including a finding with no severity field landing in a scored bucket.

Run the suite in the container:

```
docker compose exec backend python -m pytest tests/ -q
```

4. The vulnerable fixture

`backend/tests/fixtures/vulnerable_project/` is a tiny project built to make all three engines fire:

File	Deliberate vulnerability	Engine that reports it
<code>app.py</code>	SQL injection via f-string query construction (<code>SELECT ... WHERE username = '{username}'</code>)	Semgrep
<code>app.py</code>	Hardcoded password <code>admin_password_123</code> and AWS key <code>AKIAIOSFODNN7EXAMPLE</code> in source	Gitleaks
<code>.env</code>	AWS and Stripe API keys in cleartext	Gitleaks

requirements.txt	requests==2.20.0 and flask==0.12.1, both carrying known CVEs	pip-audit
------------------	--	-----------

For a larger demonstration, the demo endpoint scans OWASP Juice-Shop, a deliberately vulnerable Node.js application, which produces a realistic large finding set (hundreds of findings, score in single digits).

XIII. Operations

1. `run-svs.bat` step by step

1. **Checks the Docker daemon.** If it is not up, starts Docker Desktop and polls `docker info` once a second for up to ~90 seconds. Without this the build fails with a confusing "daemon not running" npipe error.
2. **Builds the three images** — `docker compose build backend worker frontend`. Code is baked into the images, not bind-mounted, so this is how edits take effect. It costs seconds when nothing changed, because of layer caching.
3. **Starts the stack** — `docker compose up -d`.
4. **Waits for the API health probe** — `curls http://127.0.0.1:8000/health` every 2 seconds, up to 60 tries, because the backend runs the schema migration on boot. A timeout warns but continues.
5. **Opens the browser at** `http://127.0.0.1:3000` and prints the URL table and the stop/log commands.

`start.bat` is the bare version: `docker compose up -d --build`, a 10-second wait, and the same browser open. Use it when Docker is already running and the health probe is not needed.

2. The `localhost` shadow on port 3000

Use `127.0.0.1`, not `localhost`. A VS Code PHP server already owns `[::1]:3000` on this machine, so `localhost:3000` resolves to that mock application instead of SVS. The launcher, the browser open, and the README all spell the address explicitly for this reason. This is an environment quirk of this laptop, not a property of the application.

3. Editing code: rebuild is mandatory

Backend and frontend code is copied into the images at build time. There is no volume mount of the source. Any edit to `backend/` or `frontend/` requires `docker compose build backend worker frontend` before `up -d` — which is exactly what `run-svs.bat` does. A plain `docker compose up -d` after an edit silently runs the old code. The only runtime-mounted paths are the Docker socket and `/tmp/svs_uploads`.

For frontend hot reload instead, stop the compose frontend first (port conflict), then `cd frontend && npm install && npm run dev`, which serves on `http://127.0.0.1:3000` and talks to the API at `VITE_API_URL`.

4. Logs, stops, resets

```
docker compose logs -f worker      # scan pipeline output
docker compose logs -f backend     # API output
docker compose ps                  # which containers are up
docker compose down                # stop everything; data persists
docker compose down -v             # stop and delete the database volume
```

URL	What it is
<code>http://127.0.0.1:3000</code>	React frontend (nginx)
<code>http://127.0.0.1:8000</code>	FastAPI API
<code>http://127.0.0.1:8000/docs</code>	Swagger / OpenAPI documentation
<code>http://127.0.0.1:8000/health</code>	Health check
<code>http://127.0.0.1:5432</code>	PostgreSQL (published; see XI.2)

5. Database backup and restore

`db_dummy_backup_20260921.sql` in the repository root is a full dump of the local development database, roughly 190 KB, containing the historical projects, scans, and findings created while the system was being built. Restore it into a fresh `postgres_data` volume with:

```
docker compose down -v
docker compose up -d db
# wait for pg_isready, then:
docker exec -i svb_db psql -U svb_user -d svb_database < db_dummy_backup_20260921.sql
docker compose up -d
```

Because the schema is created at backend startup by `create_all`, restore the dump *before* bringing the backend up, or the tables will already exist and the dump will error on the duplicates.

XIV. CI/CD pipeline

`.github/workflows/ci-cd.yml`, triggered on push to `main` or `develop` and on pull requests to `main`:

Job	Runs on	What it does
lint	matrix over [python, javascript]	Python: <code>pip install ruff</code> then <code>ruff check backend/</code> . JavaScript: <code>npm install</code> in frontend/ then <code>npm run lint</code> (which is <code>tsc --noEmit</code>) falling back to <code>npx eslint</code> .
test	postgres:15 and redis service containers	Sets up Python 3.11, installs <code>requirements.txt</code> plus <code>pytest</code> and <code>pytest-asyncio</code> , runs <code>pytest</code> tests/ against <code>svs_test</code> . Note the env sets <code>REDIS_URL</code> , which the application does not read (it uses <code>REDIS_HOST/REDIS_PORT</code>).
build	only on push to main, needs lint + test	Logs into GHCR and pushes the backend image as <code>ghcr.io/<repo>/backend:latest</code> and <code>:<sha></code>
deploy	only on main, needs build	POSTs to <code>secrets.RAILWAY_DEPLOY_WEBHOOK</code> . This is a stub: the comment says to replace it with the real webhook, and no frontend image is built or pushed.

Only the backend image is published. The pipeline never builds or pushes the frontend or worker images, and the `compose` file builds everything locally. Treat the registry path as plumbing for a deployment target that is not yet wired up.

XV. Known limitations and documentation drift

Read this section before trusting anything that looks like it documents itself.

Functional limitations

- **No per-stage status.** The API reports four states. The ingest view's five stage indicators and its terminal log are a timed approximation, not real progress.
- **The branch field does nothing.** `Repo.clone_from(..., depth=1)` takes no branch argument; the default branch is always what is scanned.
- **No code snippets, CWE, or CVE columns.** The findings table has no place for them, and the scanned tree is deleted at the end of the scan, so the offending line is gone by the time anyone reads the report. The UI shows the file and line and explains the absence.
- **No ownership or multi-tenancy.** Every project and scan belongs to the one local user; there is no way to partition data.
- **Triage vocabulary is closed.** Four values; anything else is a 400.
- **Dependent on the host Docker daemon.** If it is unreachable, scans fail with a plain `FAILED` and no scanner detail.
- **Archive handling is asymmetric.** The UI advertises `.tar.gz`; the API accepts only `.zip`.

Documentation drift

The in-app "Database Schema (ERD)" panel is wrong about the shipped schema. `ArchitectureDocsView.tsx`, "Database Schema (ERD)" sub-tab, describes UUID primary keys, a `role` column on users, JSONB severity counts, a `scanner` column on findings, a `user_id` FK, and "Prisma ORM". None of that exists. The real schema is in section VIII. Do not edit code against that panel.

- **The "15-Step AI Implementation Prompts" panel is the original build spec,** written before the code existed. It is accurate as provenance — it is the prompt series that produced this system — but it describes an intended system, including an authentication and tenant-isolation step that was later deliberately removed.
- **The frontend navbar reads `v2.4`** and the `package.json` reads `"version": "0.0.0"`. Neither is tracked or released; the badge is decorative.
- **The API reference panel duplicates the real routes** and is currently accurate, but it is static data in `docsAndPrompts.ts`, not generated from the live OpenAPI. Prefer `http://127.0.0.1:8000/docs`.
- **The "Download Full Project ZIP" link points at `/svs-security-scanner-project.zip`**, which nginx does not serve; it 404s.
- **`frontend-cra-legacy/` is the superseded Create React App build.** It has its own Dockerfile and nginx config and is not referenced by the compose file. It is kept for reference only.

XVI. File inventory

Every source file in the shipped system, with a one-line purpose. Paths are relative to the repository root.

Backend

Path	Purpose
backend/app/main.py	FastAPI app, CORS, table creation and the triage migration at startup
backend/app/database.py	SQLAlchemy engine, SessionLocal, get_db dependency
backend/app/api/deps.py	Local-user resolution and the RateLimiter dependency
backend/app/api/scans.py	All scan routes: dispatch, list, status, findings, triage, delete, PDF export
backend/app/api/projects.py	Project CRUD and per-project scan listing
backend/app/core/config.py	Pydantic settings with defaults
backend/app/core/celery_app.py	Celery instance, Redis broker and backend, JSON serialization
backend/app/core/rate_limit.py	Redis INCR/TTL rate limiter
backend/app/engine/ingestion.py	SSRF validation, git clone and zip extraction into per-scan directories
backend/app/engine/scanner.py	Pipeline orchestrator: runs the three scanners, scores, sets terminal status
backend/app/engine/base_scanner.py	Abstract scanner, the shared container runner, ScannerError, _save_finding
backend/app/engine/semgrep_scanner.py	Semgrep container invocation and result mapping
backend/app/engine/gitleaks_scanner.py	Gitleaks invocation, report-file readback, cleartext report deletion
backend/app/engine/dependency_scanner.py	Language detection, pip-audit and npm audit paths
backend/app/engine/normalizer.py	Severity maps and the common finding shape, with a self-check
backend/app/engine/scoring.py	The scoring formula and weight table, with a self-check
backend/app/engine/reporting.py	xhtml2pdf report template and generation
backend/app/engine/tasks.py	The run_security_scan Celery task and scan-directory cleanup
backend/app/models/base.py	Declarative base
backend/app/models/user.py	users table
backend/app/models/project.py	projects table and relationships
backend/app/models/scan.py	scans table: status, timing, score, counts, error detail
backend/app/models/finding.py	findings table including triage_status
backend/app/models/__init__.py	Registers all mappers so string relationships resolve
backend/app/schemas/project.py	ProjectCreate, ProjectOut
backend/app/schemas/scan.py	ScanOut with the four count fields
backend/app/schemas/user.py	UserCreate, UserOut — unused by any route
backend/Dockerfile	python:3.11-slim, installs build deps and git, runs uvicorn
backend/requirements.txt	13 pinned dependencies

backend/tests/test_scoring.py	Six scoring unit tests on SQLite
backend/tests/test_pipeline.py	Full-pipeline integration test against the fixture
backend/tests/fixtures/vulnerable_project/	Fixture with SQL injection, hardcoded secrets, and two vulnerable pins

Frontend

Path	Purpose
frontend/src/main.tsx	React 19 root render in StrictMode
frontend/src/App.tsx	Shell: state, polling, handlers, view switching
frontend/src/types.ts	All domain types: Scan, Finding, Severity, triage states, KB item, prompt step
frontend/src/services/api.ts	Fetch client, response interfaces, error unwrapping
frontend/src/services/adapters.ts	API-to-UI mapping: status, severity, tool, source type, counts
frontend/src/components/Navbar.tsx	Sticky nav with the five tabs, the version badge, the local-instance pill, mobile bar
frontend/src/components/DashboardView.tsx	Metrics, filters, scan list with severity pills and score badges
frontend/src/components/IngestScanView.tsx	Three ingestion modes, validation, simulated stage progress, terminal log
frontend/src/components/ScanReportView.tsx	Score gauge, filters, expandable findings, triage, JSON and PDF export
frontend/src/components/KnowledgeBaseView.tsx	Browsable vulnerability-class reference with vulnerable/secure code
frontend/src/components/ArchitectureDocsView.tsx	Architecture, prompts, API, and ERD panels — the ERD panel is stale, see XV
frontend/src/data/knowledgeBase.ts	8 static KB entries: SQLi, hardcoded secrets, deserialization, XSS, weak crypto, dependency CVEs, command injection, SSRF
frontend/src/data/docsAndPrompts.ts	Architecture overview, Mermaid topology, endpoint list, 15 build prompts
frontend/src/index.css	Tailwind v4 import and base styles
frontend/index.html	HTML shell with the root div
frontend/vite.config.ts	Plugins, path alias, HMR override for AI editors
frontend/nginx.conf	SPA fallback, hashed-asset caching with real 404s
frontend/Dockerfile	node:20 build stage, nginx serve stage, VITE_API_URL build arg
frontend/package.json	Dependencies and scripts; lint is tsc --noEmit

Root and infrastructure

Path	Purpose
docker-compose.yml	Five services, the socket and upload mounts, build args, healthcheck
run-svs.bat	One-command launcher: daemon check, build, up, health probe, browser
start.bat	Bare up --build plus browser open
.env	Deployment configuration; see X.2

<code>.github/workflows/ci-cd.yml</code>	lint, test, build, deploy jobs
<code>.gitignore</code>	Ignores <code>.env</code> , caches, <code>node_modules</code> , uploads
<code>db_dummy_backup_20260921.sql</code>	Development database dump for restore, see XIII.5
<code>README.md</code>	Quick start; the short version of sections I and XIII
<code>frontend-cra-legacy/</code>	Superseded CRA build; not in the compose file

Generated 22 September 2026 from the code as of commit `e474377` ("chore: add spec and plan"). Every statement in this document was verified against the source files listed in section XVI. Where the in-app documentation disagrees with the code, this document follows the code.